SIMATS ENGINEERING

JAVA PROGRAMMING — ASSIGNMENT 1

Campus Smart-Parking System

PART 2

Student: Doma Saketh

Register Number: 192525018

Course: Java Programming

Total Assignment Marks: 50

GitHub repository

https://github.com/sakethdoma61-11/CSA09-192525018-Assignment1/tree/main

Repository link supplied by student; verify report examples against submitted source files.

Contents

Task 7 — Override and Specialise Behaviour

Task 8 — Apply Polymorphism

Task 9 — Introduce Abstraction

Task 10 — Packages and Encapsulation

Task 11 — Implement, Test and Defend

Task 12 — Day-4 Design Challenge

Conclusion and checklist

Task 7 — Override and Specialise Behaviour (4 marks)

Override methods such as ratePerHour() when behaviour differs by vehicle category. @Override lets the compiler detect signature errors. An overriding method must not reduce visibility and must use a compatible return type. Use final only when a value, method or class must not be changed or extended.

equals() and hashCode() must agree: equal objects have equal hash codes. The supplied assignment compares registration numbers, so a Car and ElectricCar with the same registration compare equal and the example prints true. If category identity matters, include it in equality or use a different key.

```
@Override public boolean equals(Object o) {
    return o instanceof Vehicle
        && regNo.equals(((Vehicle) o).regNo);
}
@Override public int hashCode() { return regNo.hashCode(); }
```

A switch on a type field centralizes pricing but needs edits when categories are added. Overriding keeps behaviour in each class. With k vehicle types, a switch may perform up to O(k) comparisons; virtual dispatch is treated as O(1) for this analysis.

Task 8 — Apply Polymorphism (4 marks)

Overloading is compile-time polymorphism: the compiler selects calculateFee(int) or another overload based on arguments. Runtime polymorphism occurs when a Vehicle reference holds a Car or TwoWheeler object; the actual object's overridden method is selected at runtime.

```
interface Chargeable { double chargingFee(int kWh); }
abstract class Vehicle {
    private final String regNo;
    protected Vehicle(String r) { regNo = r; }
    public String getRegNo() { return regNo; }
    protected abstract double ratePerHour();
    public double calculateFee(int minutes) {
        if (minutes <= 0) return 0;
        int hours = (minutes + 59) / 60;
        return hours * ratePerHour();
    }
    @Override public boolean equals(Object o) {
        return o instanceof Vehicle && regNo.equals(((Vehicle)o).regNo);
    }
    @Override public int hashCode() { return regNo.hashCode(); }
}
class TwoWheeler extends Vehicle {
    TwoWheeler(String r) { super(r); }
    @Override protected double ratePerHour() { return 10; }
}
class Car extends Vehicle {
    Car(String r) { super(r); }
    @Override protected double ratePerHour() { return 30; }
}
class ElectricCar extends Car implements Chargeable {
    ElectricCar(String r) { super(r); }
    @Override public double chargingFee(int kWh) { return 12.0*kWh; }
}
public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = {new TwoWheeler("TN01A1"),
            new Car("TN02B2"), new ElectricCar("TN03C3")};
        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);
            if (v instanceof Chargeable c) fee += c.chargingFee(10);
            System.out.println(v.getClass().getSimpleName()+" "+
                v.getRegNo()+" -> Rs "+fee);
        }
        System.out.println(new Car("TN02B2").equals(
            new ElectricCar("TN02B2")));
    }
}
```

Expected output from the supplied assignment

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

For 130 minutes, ceiling division gives 3 hours. The fees are 3×■10 = ■30, 3×■30 = ■90, and ■90 + 10×■12 charging = ■210. The declared type is Vehicle; runtime type controls ratePerHour(). The pattern matching form `instanceof Chargeable c` requires Java 16 or later.

Task 9 — Introduce Abstraction (4 marks)

Vehicle should be abstract because a generic vehicle has common identity but no universal rate. Chargeable is an interface because charging is an optional capability. A class can extend one class but implement multiple interfaces. Interface default methods provide reusable behaviour; static methods provide related utility operations.

If Vehicle became an interface, it could not hold per-instance state initialized through constructors in the same way. If Chargeable became an abstract class, a vehicle already extending Car could not also extend it. Composition or another interface would be needed. Runtime cost is not usually the deciding factor; both designs are treated as O(1) per method call.

Task 10 — Packages and Encapsulation (3 marks)

```
parking/
  model/     Vehicle.java, ParkingTicket.java, ParkingPass.java
  facility/  ParkingFacility.java
  pricing/   PricingRule.java, HourlyPricingRule.java
  service/   ParkingService.java
  app/       Main.java
```

Use private fields, public methods for supported operations, protected members only for subclass extension and package-private members for closely related classes. Validate blank registrations, negative durations, negative kWh and invalid slot indexes. Integer wrappers are useful in collections or nullable values; compare wrapper values with equals() or unbox rather than using ==.

```
public void setDurationMinutes(int minutes) {
    if (minutes < 0)
        throw new IllegalArgumentException("Negative duration");
    this.durationMinutes = minutes;
}
```

Public fields with caller-side validation allow invalid states and force repeated checks. Each simple validation is O(1), but encapsulation avoids duplicated validation logic.

Task 11 — Implement, Test and Defend the Design (5 marks)

Integrate facility allocation, active tickets, vehicle subclasses, pass eligibility, pricing, timestamps and daily collection. A HashMap keyed by normalized registration enables average O(1) exit lookup and O(n) worst-case lookup; array/ArrayList search is O(n). Both store O(n) active tickets.

Test case	Expected behaviour	Execution evidence
Normal entry	First suitable slot allocated	Add actual result
Facility full	No slot returned	Add actual result
Release occupied slot	true; slot freed	Add actual result
Invalid/empty release	false	Add actual result
Zero-minute stay	■0 fee	Add actual result
61-minute stay	2 billable hours	Add actual result
Duplicate registration	Reject second active entry	Add actual result
Valid monthly pass	■0 parking fee	Add actual result
Unknown vehicle type	Reject safely	Add actual result

Manual daily collection example: assume a two-wheeler parks for 61 minutes at ■10 per started hour and a car parks for 60 minutes at ■30 per hour. Fees: ■20 + ■30 = ■50 total. Compare this hand calculation with the program after execution. These rates are illustrative; the uploaded assignment does not prescribe one universal final rate table.

Design defence: encapsulation protects invariants; inheritance and overriding express category-specific behaviour; polymorphism avoids long type checks; abstraction separates common identity from optional capabilities. An in-memory prototype loses data when it exits; production use would need persistence and concurrency controls. Do not label test results as actual until you run the code.

Task 12 — Day-4 Design Challenge (8 marks)

New requirements: e-bikes with a flat fee; delivery vans at double the car rate during 9 AM–5 PM; one-day visitor passes; and five floors with unequal capacities. The jagged array remains suitable. Pricing, vehicle types and pass validation are affected. If pricing is hard-coded across subclasses, new rules cause scattered edits.

Recommendation: modify the original design. Keep Vehicle, ParkingFacility, ticket handling and jagged slots. Add E-bike and DeliveryVan categories, a Pass abstraction and replaceable PricingRule implementations. This preserves tested allocation logic while isolating changing fee rules.

```
import java.time.LocalTime;
interface PricingRule {
    double fee(int hours, LocalTime entry, LocalTime exit);
}
class FlatFeeRule implements PricingRule {
    private final double amount;
    FlatFeeRule(double amount) { this.amount = amount; }
    public double fee(int h, LocalTime in, LocalTime out) {
        return amount;
    }
}
class DeliveryVanRule implements PricingRule {
    private final double carRate;
    DeliveryVanRule(double rate) { carRate = rate; }
    public double fee(int h, LocalTime in, LocalTime out) {
        boolean daytime = !in.isBefore(LocalTime.of(9,0))
                       && in.isBefore(LocalTime.of(17,0));
        return h * carRate * (daytime ? 2.0 : 1.0);
    }
}
interface Pass {
    boolean isValidFor(LocalTime entry, LocalTime exit);
}
class OneDayPass implements Pass {
    public boolean isValidFor(LocalTime in, LocalTime out) {
        return true; // Placeholder only: use LocalDateTime and stored pass date in final system.
    }
}
```

Important implementation note: the DeliveryVanRule above uses entry time as a simplifying interpretation. If the multiplier applies only to the portion of parking between 9 AM and 5 PM, split the parking interval and calculate each portion separately. The OneDayPass method shown is only an interface illustration, not production-ready validation: LocalTime has no date. A real pass must store a date/validity window and use LocalDateTime.

Acceptance tests: e-bike receives configured flat fee; van entry at 10:00 uses the doubled rate under the chosen interpretation; entry at 18:00 uses normal rate; a valid one-day pass waives the eligible fee; an expired pass does not; five unequal floor capacities allocate and release correctly. Record actual outputs after execution.

Evidence-based defence: modify rather than redesign from scratch. New subclasses and pricing strategies can be added while retaining the allocation class. Trade-offs are extra classes and the need to configure rules correctly. Support the final decision with executed tests and a count of changed source files.

Conclusion and Submission Checklist

The smart-parking system benefits from clear class responsibilities, jagged arrays for unequal floor capacities, encapsulated fields, an abstract Vehicle class, capability interfaces and replaceable pricing rules. Before submission: compile and run required programs; add genuine output screenshots; fill in the real GitHub URL; include README.md with student details, Java version, run steps and task summaries; commit progressively; and keep each PDF under 10 MB.

Primary source: SIMATS Online Programming Lab, Java Programming, Assignment 1 — “Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System,” uploaded PDF, pages 1–17. Task titles, expected outputs, marks and submission requirements are based on this source.

JAVA ASSIGNMENT 1 — PART 2

Task07 — execution output

Java terminal output — Task07

```
Command: javac *.java  |  java Task07Demo

> Vehicle[TN02B2]
> Same registration equals: true
> Hash codes equal: true
```

Output captured from a successful local compile/run of the included demo source.

Demo-program output; not a screenshot from the student's own computer.

JAVA ASSIGNMENT 1 — PART 2

Task08 — execution output

Java terminal output — Task08

```
Command: javac *.java  |  java FeeDemo

> TwoWheeler TN01A1 -> Rs 30.0
> Car TN02B2 -> Rs 90.0
> ElectricCar TN03C3 -> Rs 210.0
> true
```

Output captured from a successful local compile/run of the included demo source.

Demo-program output; not a screenshot from the student's own computer.

JAVA ASSIGNMENT 1 — PART 2

Task09 — execution output

Java terminal output — Task09

```
Command: javac *.java  |  java Task09Demo

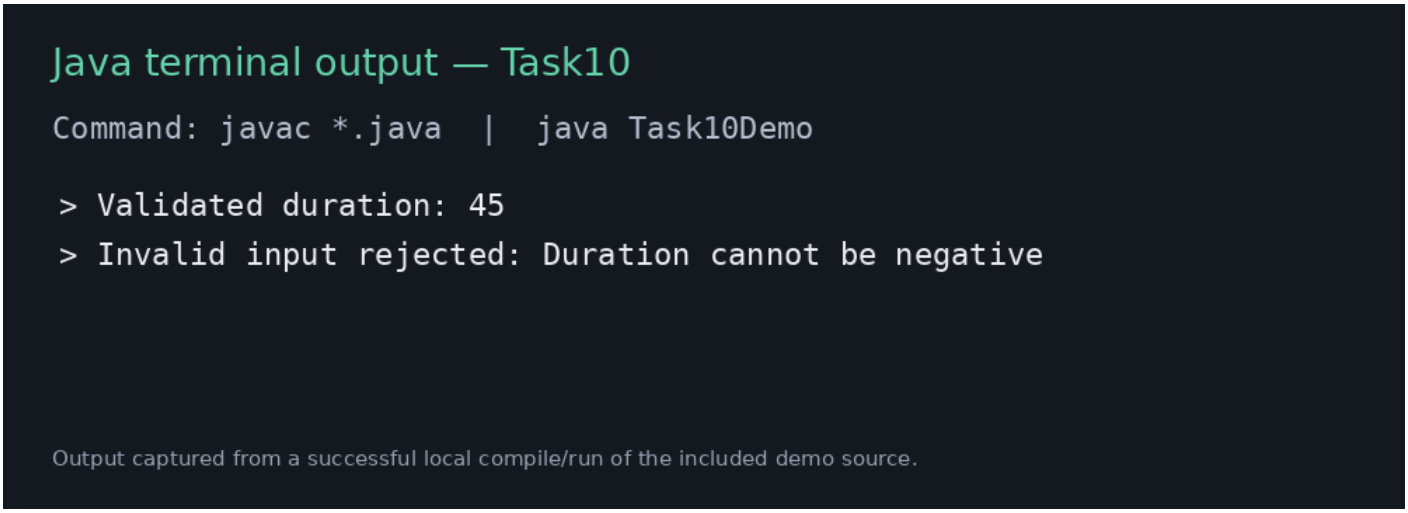
> Parking rate: Rs 30.0
> Charging for 5 kWh: Rs 60.0
```

Output captured from a successful local compile/run of the included demo source.

Demo-program output; not a screenshot from the student's own computer.

JAVA ASSIGNMENT 1 — PART 2

Task10 — execution output



Demo-program output; not a screenshot from the student's own computer.

JAVA ASSIGNMENT 1 — PART 2

Task11 — execution output

Java terminal output — Task11

```
Command: javac *.java  |  java Task11Demo

> 0 minutes -> 0 billable hours
> 60 minutes -> 1 billable hours
> 61 minutes -> 2 billable hours
> 130 minutes -> 3 billable hours
> Illustrative daily total: Rs 50.0
```

Output captured from a successful local compile/run of the included demo source.

Demo-program output; not a screenshot from the student's own computer.

JAVA ASSIGNMENT 1 — PART 2

Task12 — execution output

Java terminal output — Task12

```
Command: javac *.java  |  java Task12Demo

> E-bike flat fee: Rs 20.0
> Van at 10:00, 2 hours: Rs 120.0
> Van at 18:00, 2 hours: Rs 60.0
> Note: van rule uses entry time as an explicit simplifying assumption.
```

Output captured from a successful local compile/run of the included demo source.

Demo-program output; not a screenshot from the student's own computer.